



SAIR

Spatial AI & Robotics Lab

CSE 473/573

L11: ALIGNMENT & FITTING

Chen Wang

Spatial AI & Robotics Lab

Department of Computer Science and Engineering



University at Buffalo The State University of New York

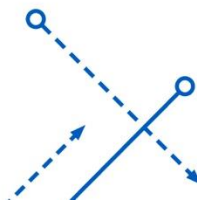
What are Alignment and Fitting?

- Alignment

- Find the parameters of a transformation that best aligns matched points

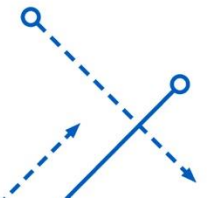
- Fitting

- Find the parameters of a model that best fit the data

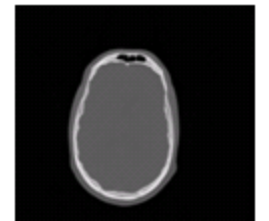
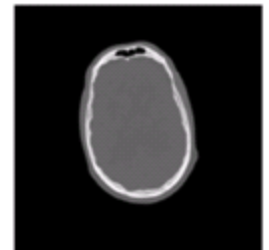
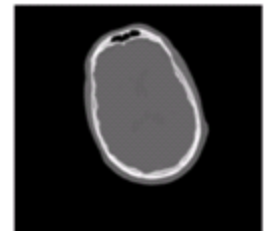
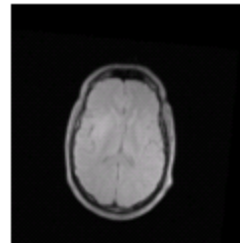
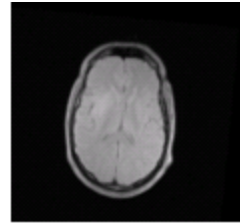
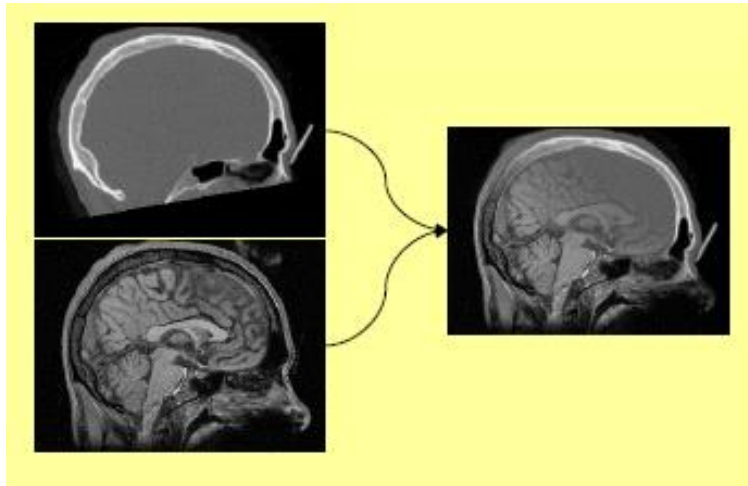


Fitting and Alignment: Methods

- Hypothesize and test
 - Generalized Hough transform
- **General Alignment**
 - **Homographies**
 - **Rotational Panoramas**
 - Global Alignment
 - RANSAC
 - Warping
 - Blending
- Global optimization / Search for parameters
 - Least squares fit
 - Robust least squares
 - Other parameter search methods



Motivation: Medical image registration



Motivation

- Getting the whole picture
 - Typical camera: $50^\circ \times 35^\circ$



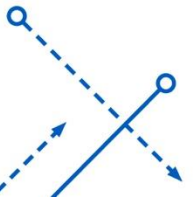
Motivation

- Getting the whole picture
 - Typical camera: $50^\circ \times 35^\circ$
 - Human Vision: $176^\circ \times 135^\circ$



Motivation

- Getting the whole picture
 - Typical camera: $50^\circ \times 35^\circ$
 - Human Vision: $176^\circ \times 135^\circ$



Alignment

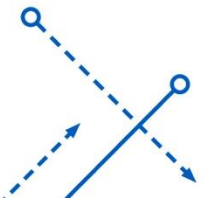
- Homography
- Rotational Panoramas
- RANSAC (Next Lecture)
- Global alignment
- Warping
- Blending



(a)



(b)



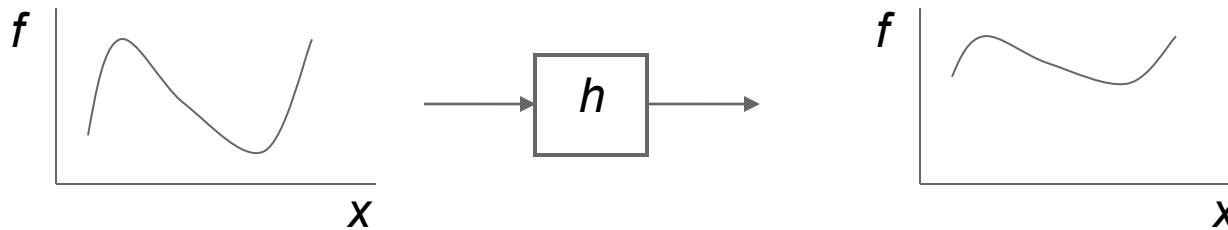
Motion models

- What happens when we take two images with a camera and try to align them?
- translation?
- rotation?
- scale?
- affine?
- perspective?



Image Warping (Recap)

- image filtering: change *range* of image
 - $g(x) = h(f(x))$



- image warping: change *domain* of image
 - $g(x) = f(h(x))$

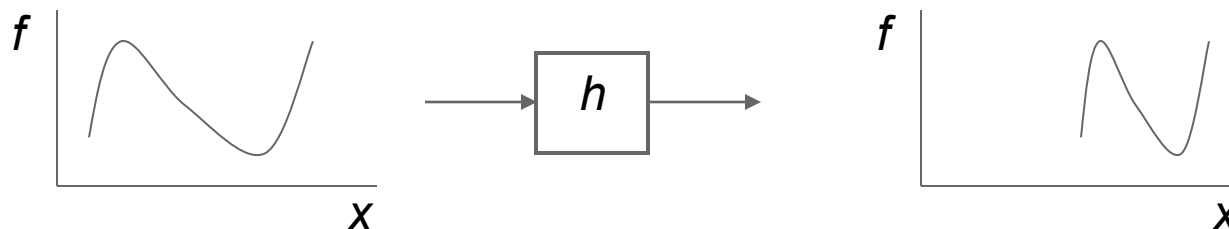
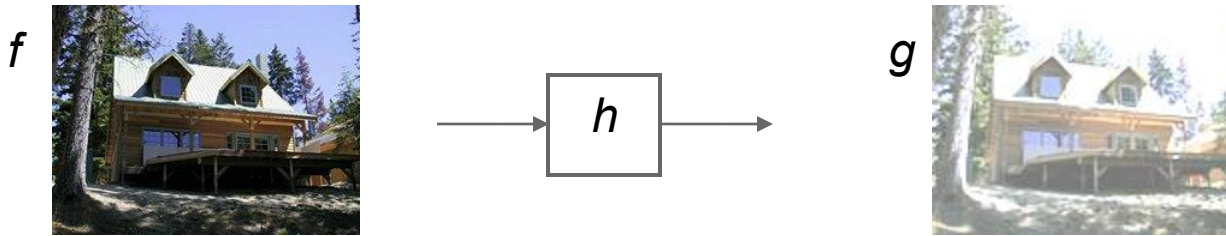
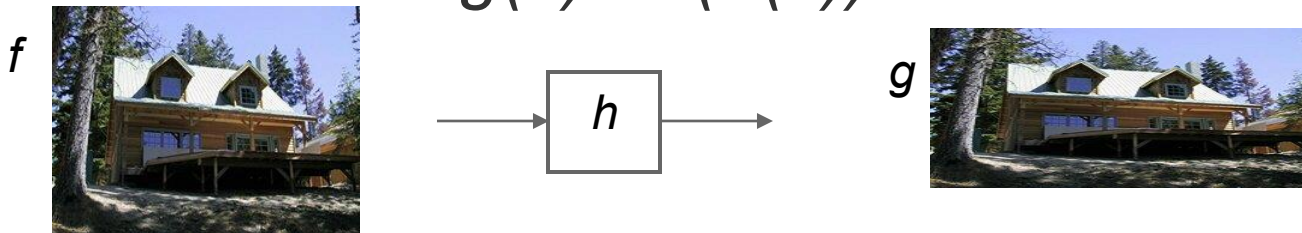


Image Warping

- image filtering: change *range* of image
 - $g(x) = h(f(x))$



- image warping: change *domain* of image
 - $g(x) = f(h(x))$



Parametric (global) warping

- Examples of parametric warps:



translation



rotation



aspect



affine



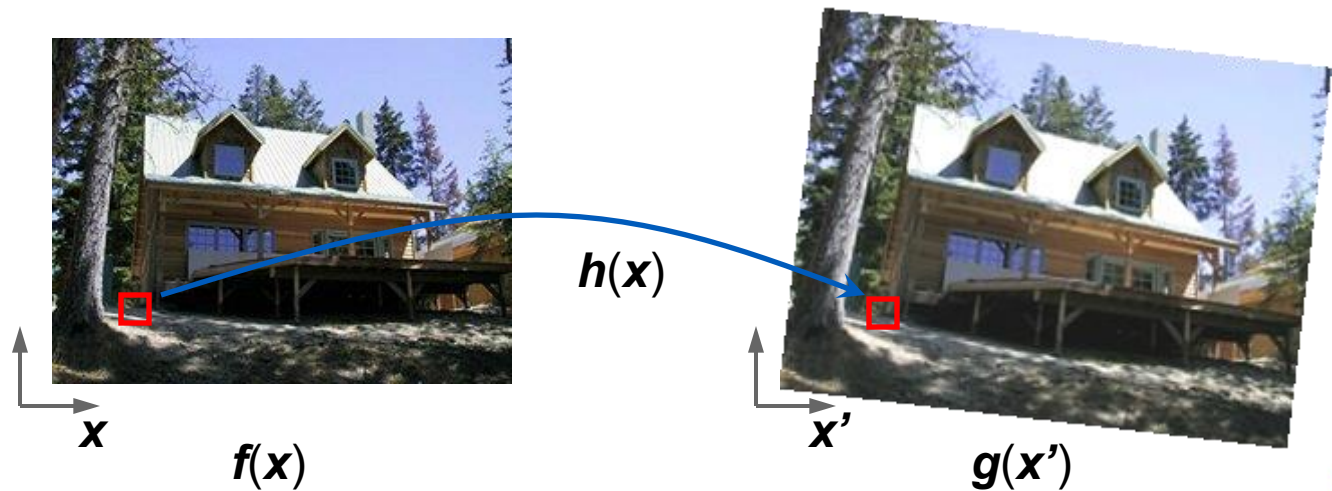
perspective



cylindrical

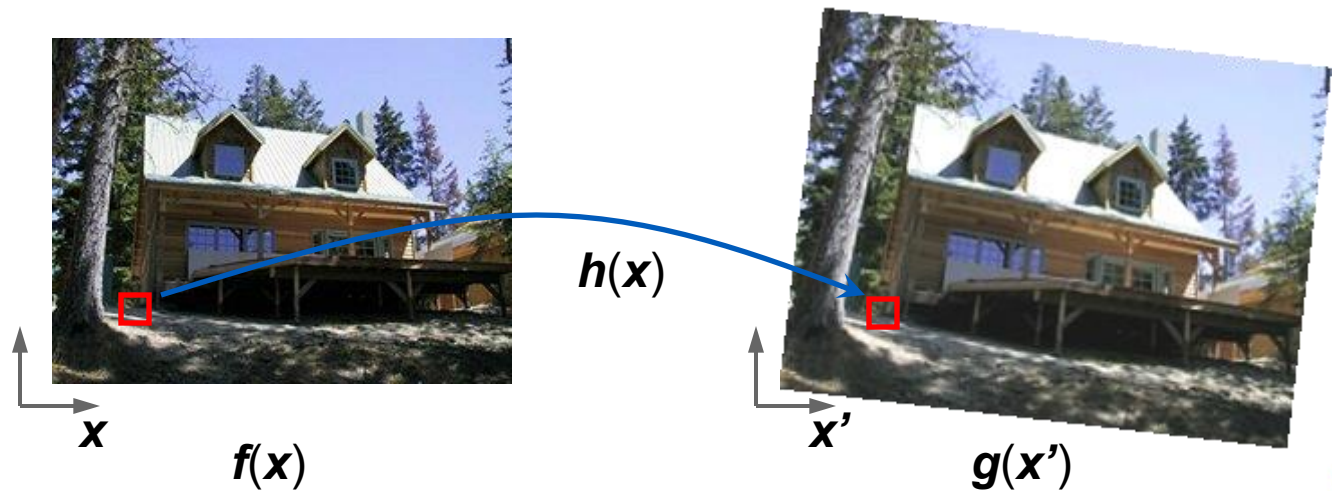
Image Warping

- Given a coordinate transform $\mathbf{x}' = \mathbf{h}(\mathbf{x})$ and a source image $\mathbf{f}(\mathbf{x})$, how do we compute a transformed image $\mathbf{g}(\mathbf{x}') = \mathbf{f}(\mathbf{h}(\mathbf{x}))$?



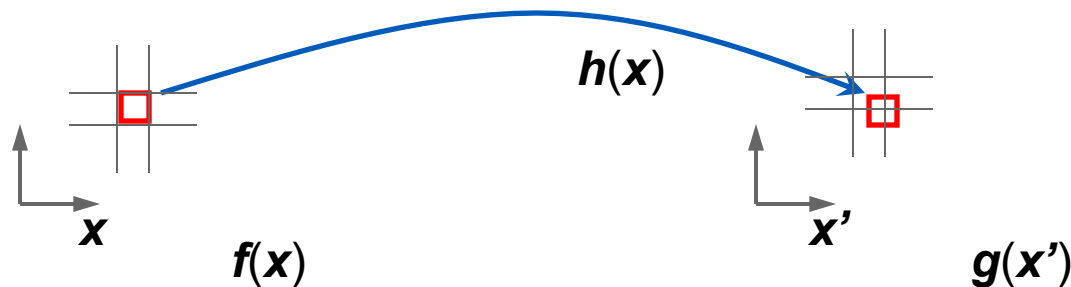
Forward Warping

- Send each pixel $f(\mathbf{x})$ to its corresponding location $\mathbf{x}' = h(\mathbf{x})$ in $g(\mathbf{x}')$
- What if pixel lands “between” two pixels?



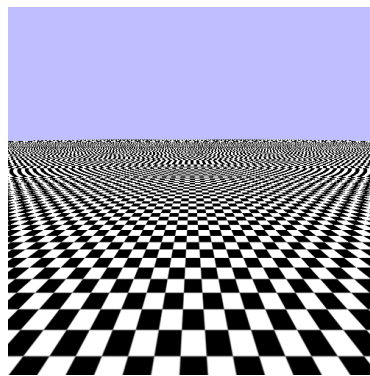
Forward Warping

- Send each pixel $f(\mathbf{x})$ to its corresponding location $\mathbf{x}' = h(\mathbf{x})$ in $g(\mathbf{x}')$
- What if pixel lands “between” two pixels?
- Answer: add “contribution” to several pixels, normalize later

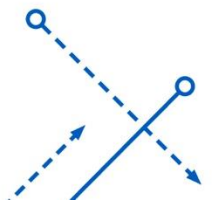
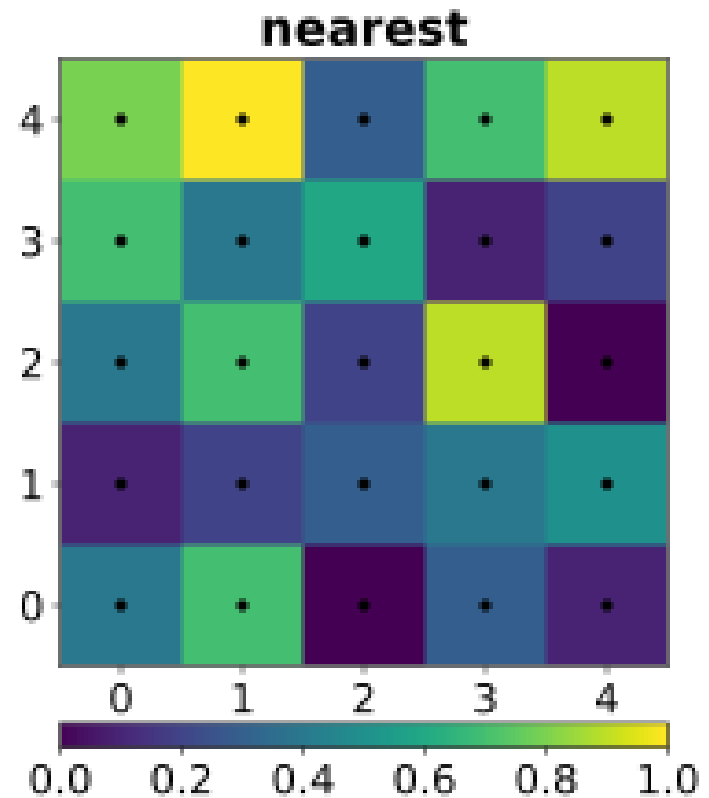
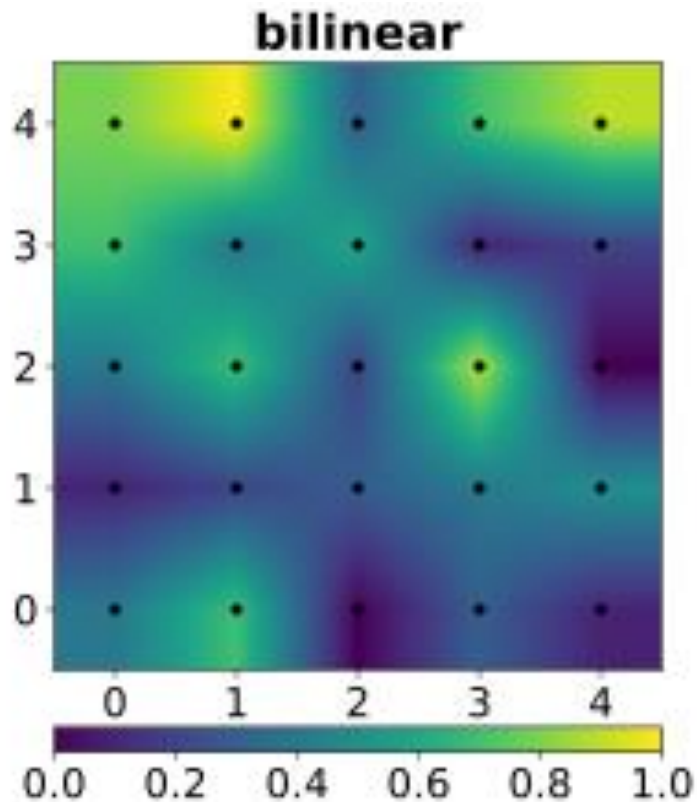


Interpolation

- Possible interpolation filters:
 - **Nearest Neighbor**
 - **Bilinear**
 - **Bicubic**
- Needed to prevent “jaggies” and “texture crawl”

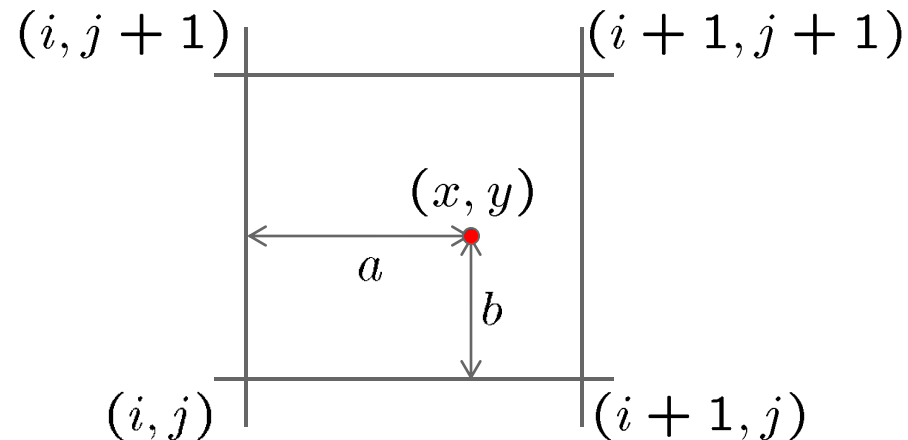


Bilinear interpolation



Bilinear interpolation

Sampling at $f(x,y)$:

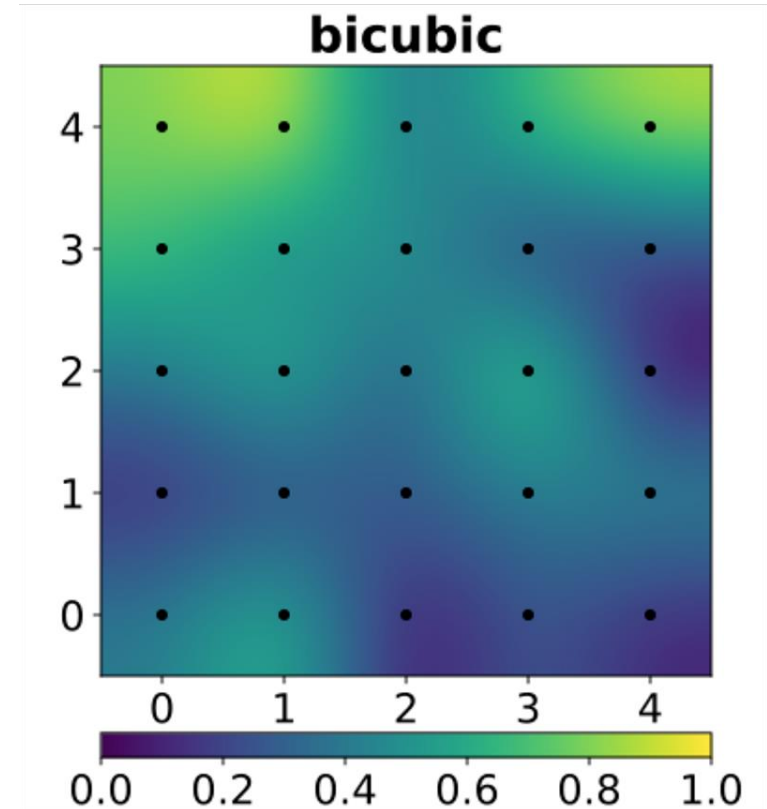
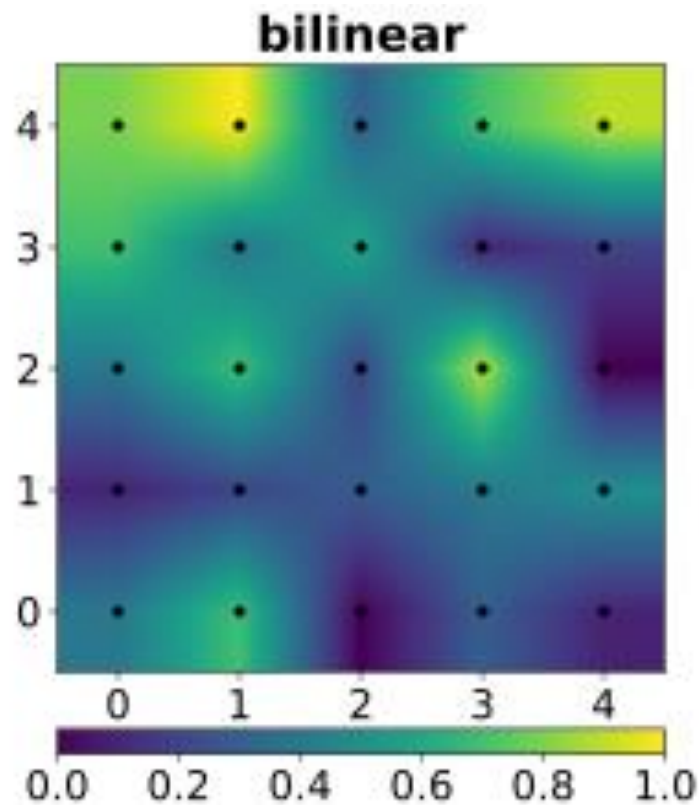


$$\begin{aligned} f(x, y) = & (1 - a)(1 - b) f[i, j] \\ & + a(1 - b) f[i + 1, j] \\ & + ab f[i + 1, j + 1] \\ & + (1 - a)b f[i, j + 1] \end{aligned}$$

Summation of Weights is 1

Bicubic interpolation

- Bilinear interpolation take pixel intensities into account.
- Bicubic interpolation also takes image gradients into account.



If the quality is of concern, bicubic would be the best choice.

Bicubic interpolation

$$p(x, y) = \sum_{i=0}^3 \sum_{j=0}^3 a_{ij} x^i y^j.$$

surface $p(x, y)$ on the **unit square** $[0, 1] \times [0, 1]$ that is continuous

This requires determining the 16 coefficients.

Consider 4 corners of the unit square. $(0, 0)$ $(1, 0)$ $(0, 1)$ $(1, 1)$

1. $f(0, 0) = p(0, 0) = a_{00},$
2. $f(1, 0) = p(1, 0) = a_{00} + a_{10} + a_{20} + a_{30},$
3. $f(0, 1) = p(0, 1) = a_{00} + a_{01} + a_{02} + a_{03},$
4. $f(1, 1) = p(1, 1) = \sum_{i=0}^3 \sum_{j=0}^3 a_{ij}.$

Bicubic interpolation

We need following derivatives

$$p_x(x, y) = \sum_{i=1}^3 \sum_{j=0}^3 a_{ij} i x^{i-1} y^j,$$

$$p_y(x, y) = \sum_{i=0}^3 \sum_{j=1}^3 a_{ij} x^i j y^{j-1},$$

$$p_{xy}(x, y) = \sum_{i=1}^3 \sum_{j=1}^3 a_{ij} i x^{i-1} j y^{j-1}.$$

Likewise, eight equations for the derivatives in the x and the y directions:

1. $f_x(0, 0) = p_x(0, 0) = a_{10},$
2. $f_x(1, 0) = p_x(1, 0) = a_{10} + 2a_{20} + 3a_{30},$
3. $f_x(0, 1) = p_x(0, 1) = a_{10} + a_{11} + a_{12} + a_{13},$
4. $f_x(1, 1) = p_x(1, 1) = \sum_{i=1}^3 \sum_{j=0}^3 a_{ij} i,$
5. $f_y(0, 0) = p_y(0, 0) = a_{01},$
6. $f_y(1, 0) = p_y(1, 0) = a_{01} + a_{11} + a_{21} + a_{31},$
7. $f_y(0, 1) = p_y(0, 1) = a_{01} + 2a_{02} + 3a_{03},$
8. $f_y(1, 1) = p_y(1, 1) = \sum_{i=0}^3 \sum_{j=1}^3 a_{ij} j.$

And four equations for the xy mixed partial derivative:

1. $f_{xy}(0, 0) = p_{xy}(0, 0) = a_{11},$
2. $f_{xy}(1, 0) = p_{xy}(1, 0) = a_{11} + 2a_{21} + 3a_{31},$
3. $f_{xy}(0, 1) = p_{xy}(0, 1) = a_{11} + 2a_{12} + 3a_{13},$
4. $f_{xy}(1, 1) = p_{xy}(1, 1) = \sum_{i=1}^3 \sum_{j=1}^3 a_{ij} ij.$

Bicubic interpolation

Grouping the unknown parameters a_{ij} in a vector

$$\alpha = [a_{00} \ a_{10} \ a_{20} \ a_{30} \ a_{01} \ a_{11} \ a_{21} \ a_{31} \ a_{02} \ a_{12} \ a_{22} \ a_{32} \ a_{03} \ a_{13} \ a_{23} \ a_{33}]^T$$

and letting

$$x = [f(0,0) \ f(1,0) \ f(0,1) \ f(1,1) \ f_x(0,0) \ f_x(1,0) \ f_x(0,1) \ f_x(1,1) \ f_y(0,0) \ f_y(1,0) \ f_y(0,1) \ f_y(1,1) \ f_{xy}(0,0) \ f_{xy}(1,0) \ f_{xy}(0,1) \ f_{xy}(1,1)]^T,$$

the above system of equations can be reformulated into a matrix for the linear equation $A\alpha = x$.

Inverting the matrix gives the more useful linear equation $A^{-1}x = \alpha$, where

$$A^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 3 & 0 & 0 & -2 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 2 & -2 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -3 & 3 & 0 & 0 & -2 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & -2 & 0 & 0 & 1 & 1 & 0 & 0 \\ -3 & 0 & 3 & 0 & 0 & 0 & 0 & 0 & -2 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -3 & 0 & 3 & 0 & 0 & 0 & 0 & 0 & -2 & 0 & -1 & 0 \\ 9 & -9 & -9 & 9 & 6 & 3 & -6 & -3 & 6 & -6 & 3 & -3 & 4 & 2 & 2 & 1 \\ -6 & 6 & 6 & -6 & -3 & -3 & 3 & 3 & -4 & 4 & -2 & 2 & -2 & -2 & -1 & -1 \\ 2 & 0 & -2 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 2 & 0 & -2 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ -6 & 6 & 6 & -6 & -4 & -2 & 4 & 2 & -3 & 3 & -3 & 3 & -2 & -1 & -2 & -1 \\ 4 & -4 & -4 & 4 & 2 & 2 & -2 & -2 & 2 & -2 & 2 & -2 & 1 & 1 & 1 & 1 \end{bmatrix},$$

which allows α to be calculated quickly and easily.

A more compact form:

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -3 & 3 & -2 & -1 \\ 2 & -2 & 1 & 1 \end{bmatrix} \begin{bmatrix} f(0,0) & f(0,1) & f_y(0,0) & f_y(0,1) \\ f(1,0) & f(1,1) & f_y(1,0) & f_y(1,1) \\ f_x(0,0) & f_x(0,1) & f_{xy}(0,0) & f_{xy}(0,1) \\ f_x(1,0) & f_x(1,1) & f_{xy}(1,0) & f_{xy}(1,1) \end{bmatrix} \begin{bmatrix} 1 & 0 & -3 & 2 \\ 0 & 0 & 3 & -2 \\ 0 & 1 & -2 & 1 \\ 0 & 0 & -1 & 1 \end{bmatrix},$$

Panoramas

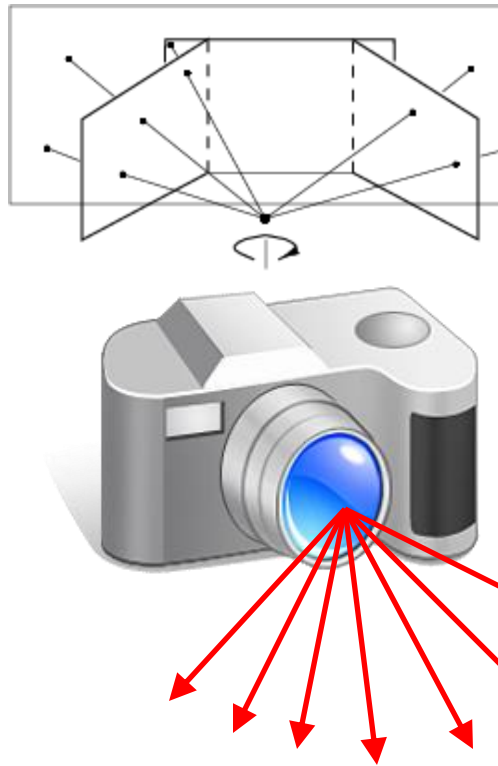


image from S. Seitz

Obtain a wide angle view by combining multiple images.

How to stitch together a panorama?

- Basic Procedure

- Take a sequence of images from the same position
 - Rotate the camera about its **optical center**.
 - Compute transformation between 2nd and 1st image
 - Transform the 2nd image to overlap with the 1st
 - Blend the two together to create a mosaic
 - (If there are more images, repeat)
- Why should this work at all?
 - Why do we rotate w.r.t. optical center?

Correspondence

- Allows us to map image back to some real space

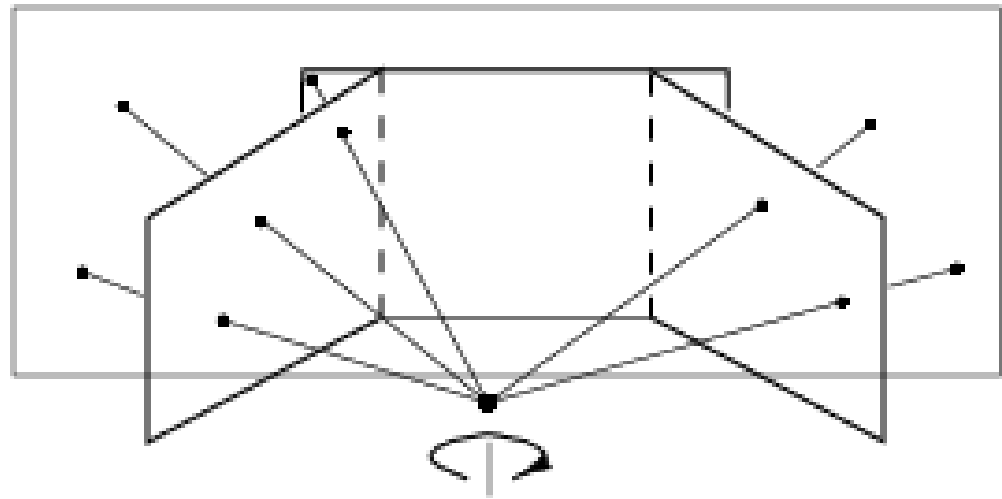
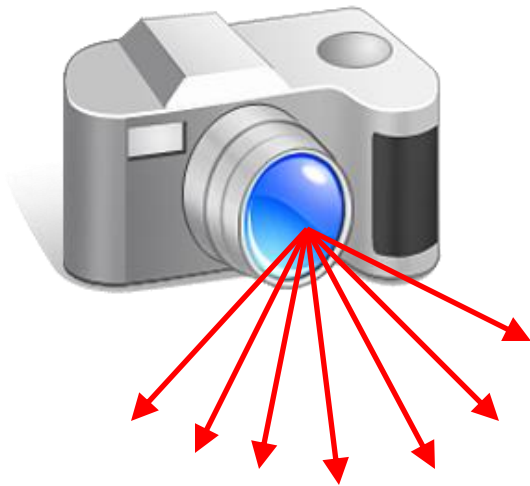
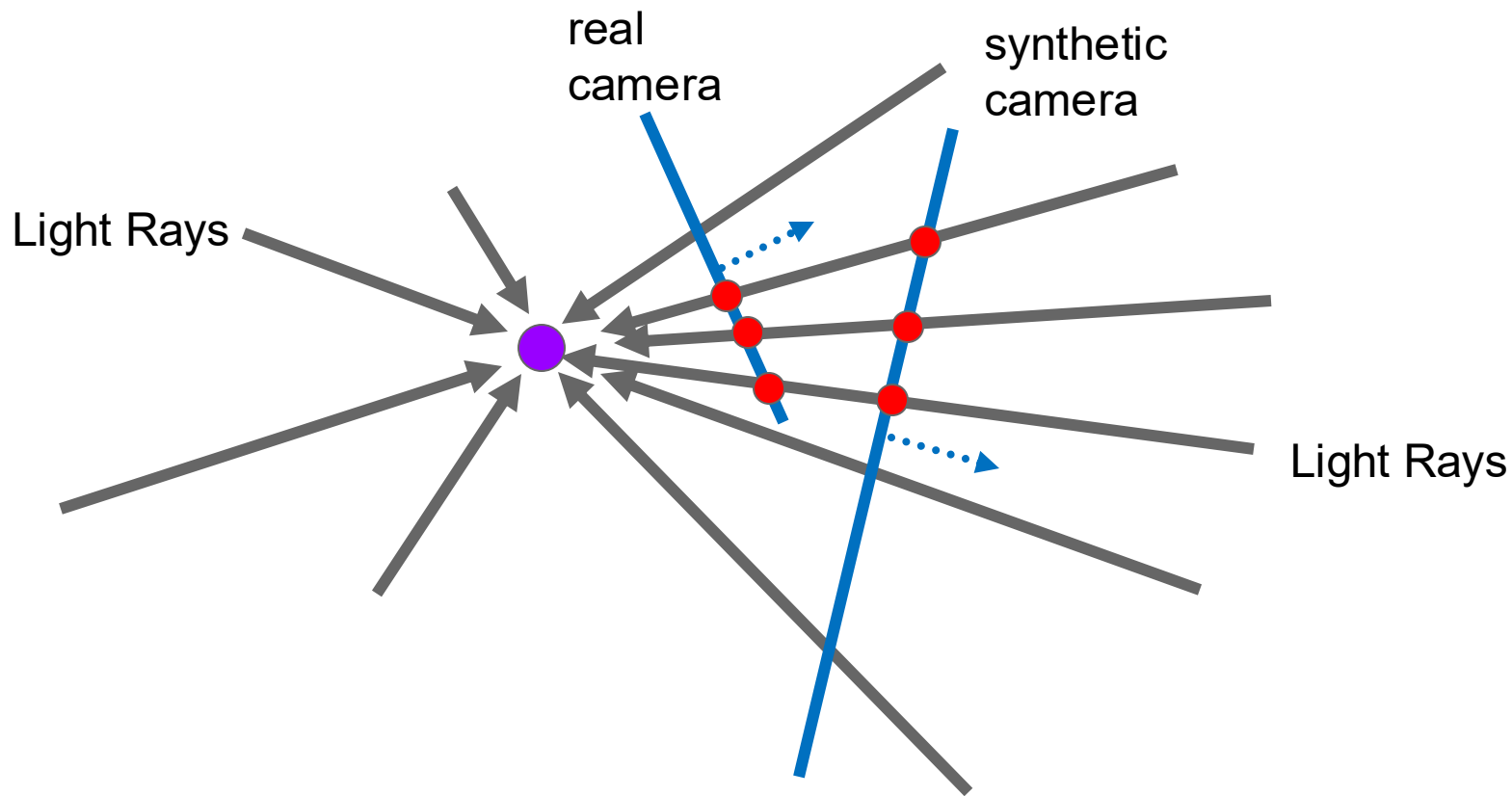


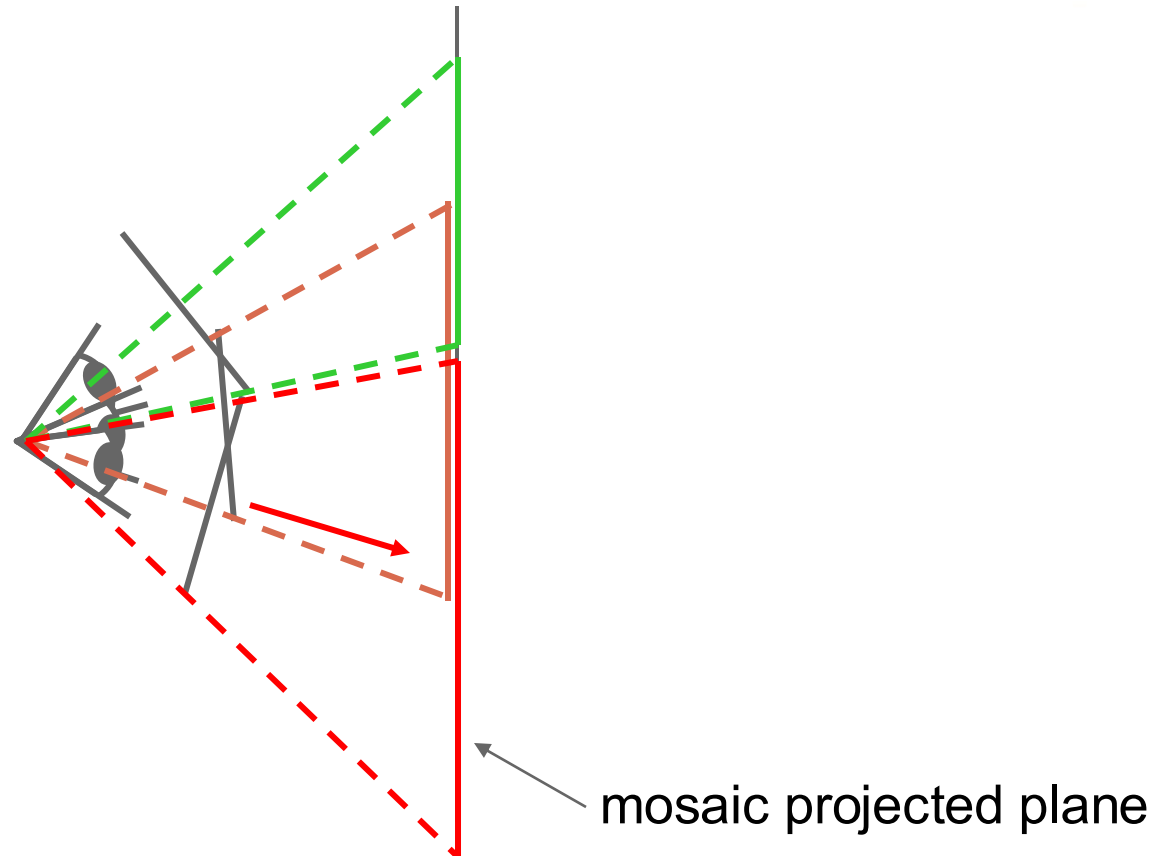
image from S. Seitz

Panoramas: generating synthetic views



Can generate any synthetic camera view
as long as it has **the same center of projection!**

Image reprojection



- The mosaic has a natural interpretation in 3D
 - The images are reprojected onto a common plane
 - The mosaic is formed on this plane
 - Mosaic is a *synthetic wide-angle camera*

Recap: 2D coordinate transformations

- translation: $\mathbf{x}' = \mathbf{x} + \mathbf{t}$ $\mathbf{x} = (x, y)$
- rotation: $\mathbf{x}' = \mathbf{R} \mathbf{x} + \mathbf{t}$
- similarity: $\mathbf{x}' = s \mathbf{R} \mathbf{x} + \mathbf{t}$
- affine: $\mathbf{x}' = \mathbf{A} \mathbf{x} + \mathbf{t}$
- perspective: $\underline{\mathbf{x}}' \cong \mathbf{H} \underline{\mathbf{x}}$ $\underline{\mathbf{x}} = (x, y, 1)$
($\underline{\mathbf{x}}$ is a *homogeneous* coordinate)
- These all form a nested *group* (closed w/ inv.)

Recap: Basic 2D Transformations

- Basic 2D transformations as 3x3 matrices

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Translate

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Scale

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

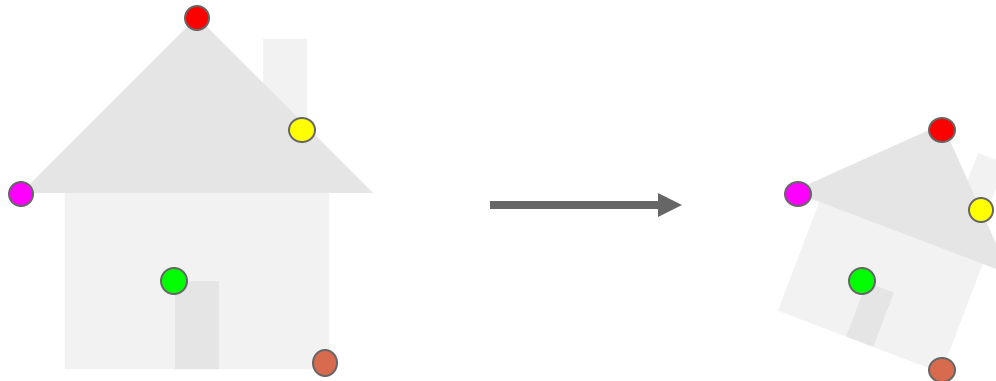
Rotate

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Shear (Skew)

Image alignment

- Two broad approaches:
 - Direct (pixel-based) alignment
 - Search for alignment where most pixels agree
 - Feature-based alignment
 - Search for alignment where *extracted features* agree
 - Can be verified using pixel-based alignment



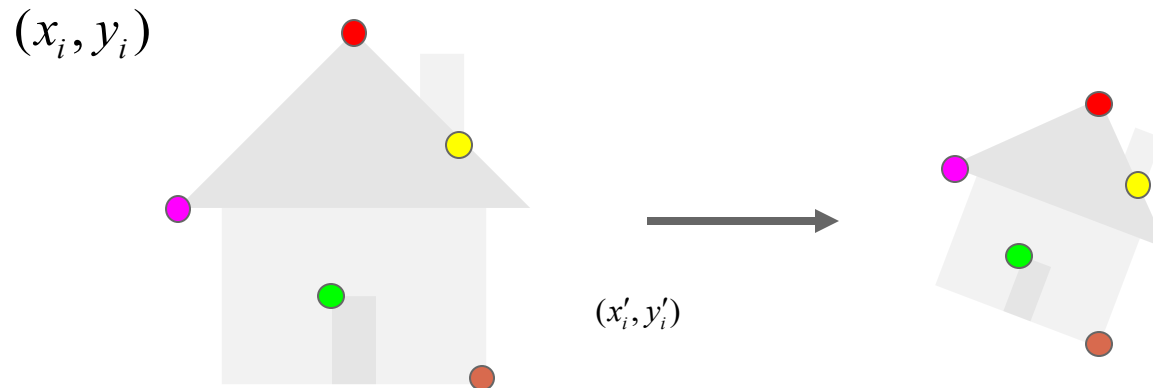
Fitting an affine transformation



Affine model approximates perspective projection of planar objects.

Fitting an affine transformation

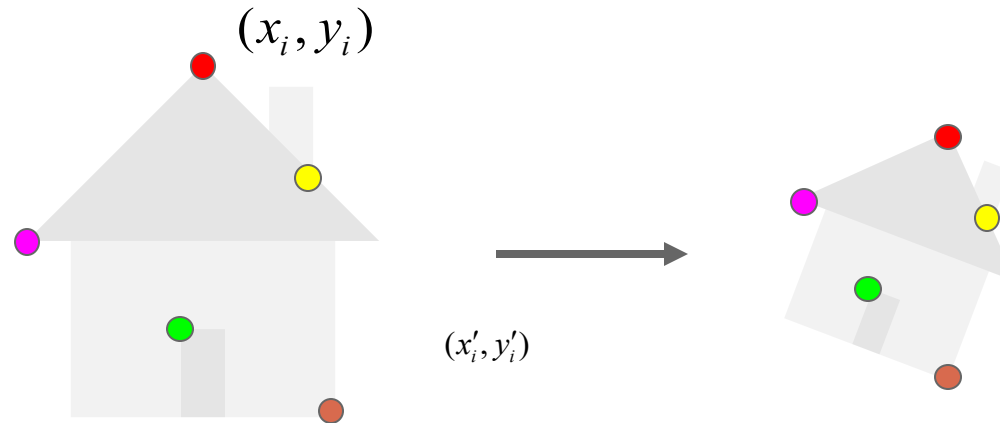
- Assuming we know the correspondences, how do we get the transformation?



$$\begin{bmatrix} x'_i \\ y'_i \end{bmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} + \begin{bmatrix} t_1 \\ t_2 \end{bmatrix}$$

Fitting an affine transformation

- Assuming we know the correspondences, how do we get the transformation?



$$\begin{bmatrix} x'_i \\ y'_i \end{bmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} + \begin{bmatrix} t_1 \\ t_2 \end{bmatrix}$$

$$\begin{bmatrix}$$

$$\begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} \end{bmatrix}$$

Fitting an affine transformation

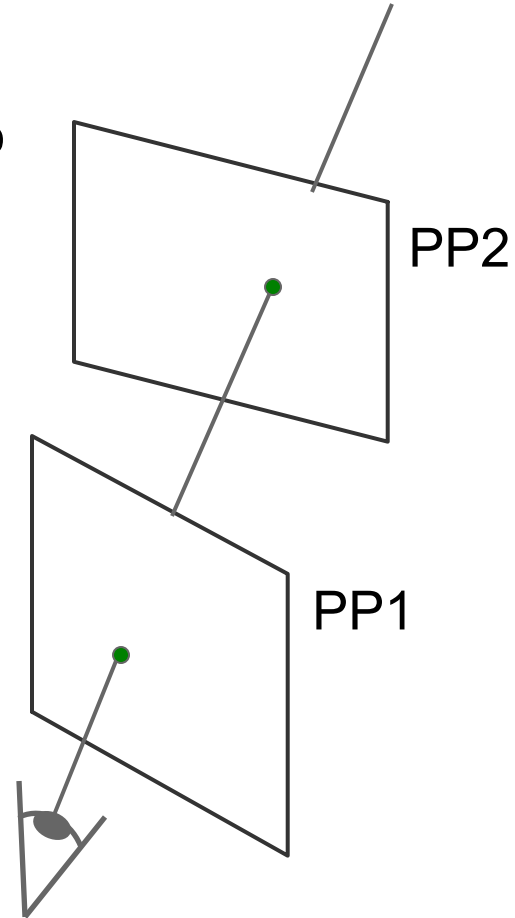
$$\begin{bmatrix} \dots & & & & & \\ x_i & y_i & 0 & 0 & 1 & 0 \\ 0 & 0 & x_i & y_i & 0 & 1 \\ \dots & & & & & \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} \dots \\ x'_i \\ y'_i \\ \dots \end{bmatrix}$$

- How many matches (correspondence pairs) do we need to solve for the transformation parameters?
- Once we have solved for the parameters, how do we compute the coordinates of the corresponding point for (x_{new}, y_{new}) ?

Homography

- Take as a 2D **image warp** using projective transform.
- A **projective transform** is a mapping between any two PPs with the same center of projection
 - rectangle should map to arbitrary quadrilateral
 - parallel lines aren't preserved.
 - but straight lines are preserved.
- Called **Homography**

$$\begin{bmatrix} wx' \\ wy' \\ w \\ \mathbf{p}' \end{bmatrix} = \begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \\ \mathbf{H} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \\ \mathbf{p} \end{bmatrix}$$



Solving for homographies

(x, y)



$$\begin{pmatrix} wx'/w & wy'/w \end{pmatrix} = (x', y')$$

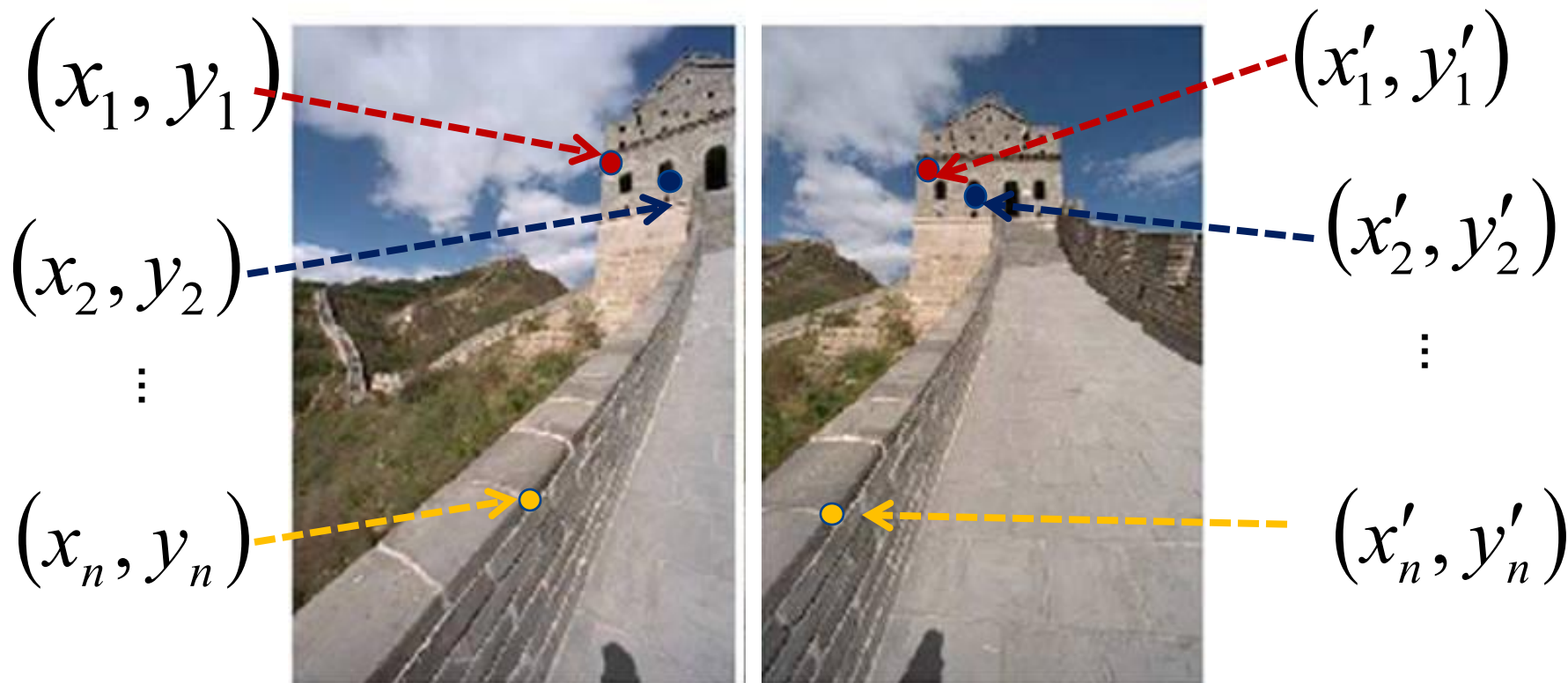
To **apply** a given homography $\mathbf{H}$

- Compute $\mathbf{p}' = \mathbf{H}\mathbf{p}$ (regular matrix multiply)
- Convert $\mathbf{p}'$ from homogeneous to image coordinates

$$\begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix} = \begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$\mathbf{p}'$ $\mathbf{H}$ $\mathbf{p}$

Solving for homographies



To **compute** the homography given pairs of corresponding points, we need to set up an equation where the parameters of **H** are the unknowns...

Solving for homographies

$$\mathbf{p}' = \mathbf{H}\mathbf{p} \quad \begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Can set scale factor $i = 1$ or $\|\mathbf{H}\| = 1$. So, there are 8 unknowns.
- Set up a system of linear equations:

$$\bullet \mathbf{A}\mathbf{h} = \mathbf{b}$$

where vector of unknowns $\mathbf{h} = [a, b, c, d, e, f, g, h]^T$

- Need at least 8 equations (4 points), but the more the better...
- Solve for $\mathbf{H}$. If over-constrained, solve using least-squares:

$$\min \|\mathbf{A}\mathbf{h} - \mathbf{b}\|^2$$

$$\mathbf{h} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}$$

Proof of least squares

- $F(h) = ||Ah - b||^2 = (Ah - b)^T (Ah - b)$
- $F(h) = h^T A^T Ah - h^T A^T b - b^T Ah + b^T b$
- $\frac{\partial}{\partial h} F(h) = 2A^T Ah - A^T b - (b^T A)^T$
- Setting derivative to 0: $\frac{\partial}{\partial h} F(h) = 0$
- $A^T Ah = A^T b$
- $h = (A^T A)^{-1} A^T b$

How to stitch together a panorama?

- Take a sequence of images from the same position
 - Rotate the camera about its optical center
- Compute transformation between second image and first
- Transform the second image to overlap with the first
- Blend the two together to create a mosaic
- If there are more images, repeat

Content

- Stitching
 - Alignment
 - Interpolation
 - Homography
 - Fitting
 - Solving for homographies

